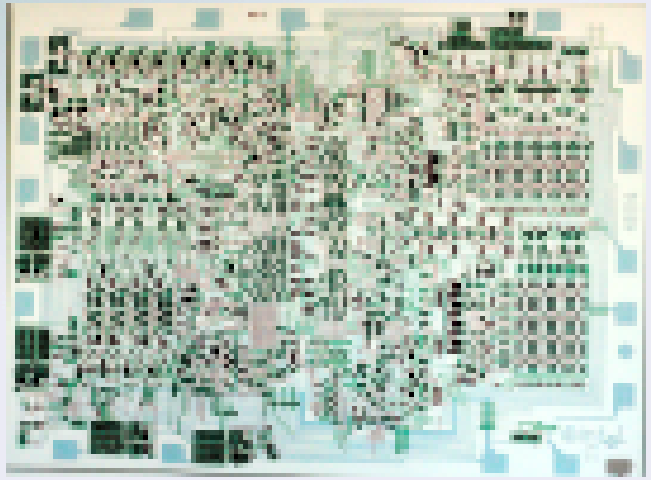


Lecture 8a: Programming the LC-3 in Assembly Language

CSCI11: **Computer Architecture and Organization**

Readings

- *Chapter 8* Patt and Patel: Introduction to Computing Systems...
- *Guide to Using LC-3 Tools* - *Instructor Repo*
- *Appendix A* Patt and Patel: Introduction to Computing Systems... - *Instructor Repo*

Syllabus Updates

Date	Week	Content
3/9	6	Cookbook and Programming
3/16	7	The Stack and Programming
3/23	-	Spring Break
3/30	8	**RPN and SMC**
4/5	9	TRAPS and Complete SMC
4/12	10	Test 2

Monday Class - Check in Notes

1. Make sure you *Commit/Sync* often, it saves your work and updates your homework in GitHub
2. You may edit code IN LC3Tools, however, use VS Code to save your code!

AI Issues

1. LC-3 Assembly language is a bit esoteric, which means an LLM won't have a complete grasp of it
2. Microsoft's CoPilot is too aggressive, which means it regularly recommends stuff which flat out **does not work**

So what to do?

Continue to develop critical thinking skills which help you to understand where AI might be successful (or accurate) and where it will have issues, this is where you will make money.

Postfix Notation (*Reverse Polish Notation*)

Infix: ***((5 + 4) * 9) / (6-3) =***

Postfix: ***5 4 + 9 * 6 3 - /***

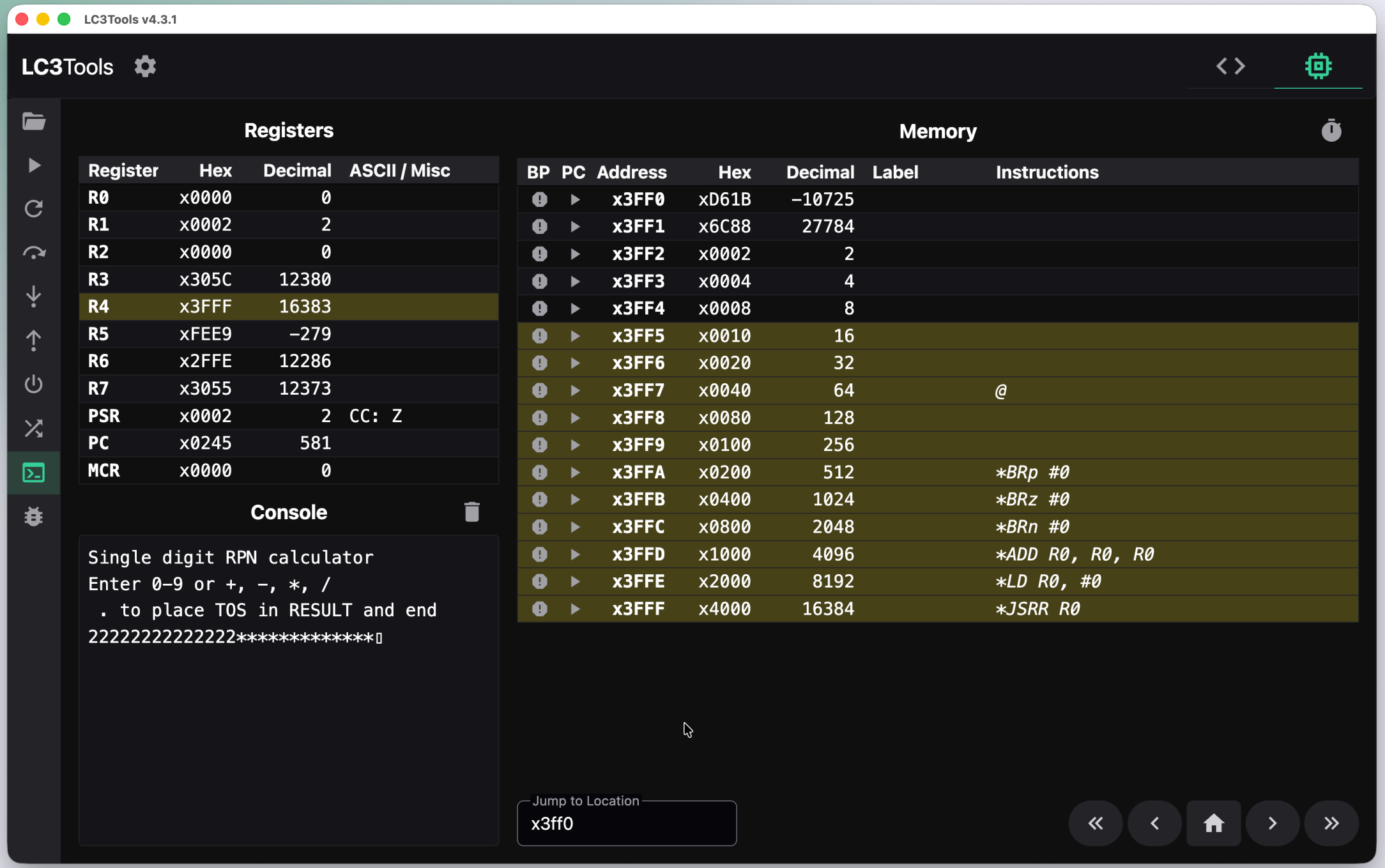
Postfix: *5 4 + 9 * 6 3 - /*

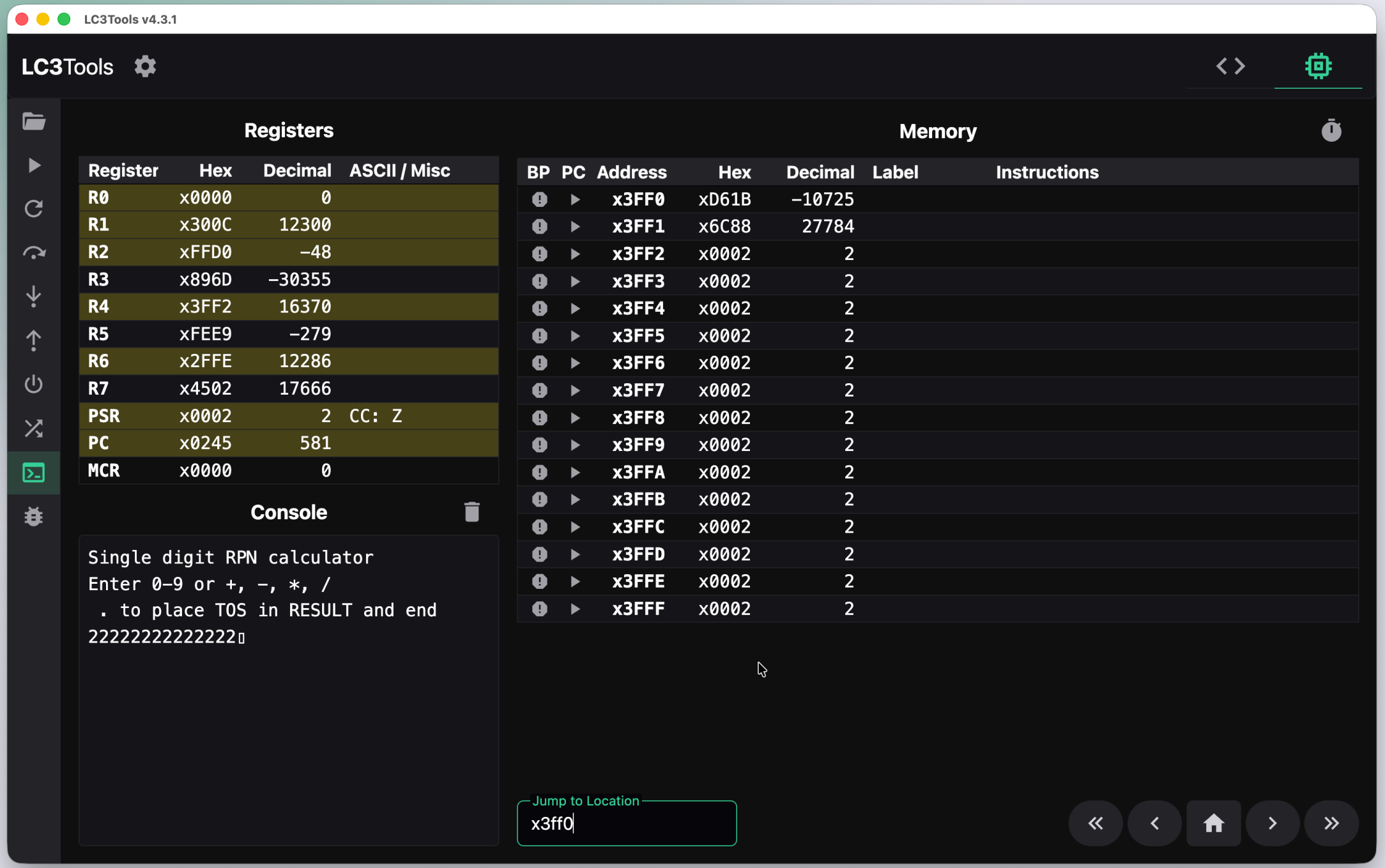
1. Put a number on the stack - 5
2. Put a number on the stack - 4
3. Put a operand (+, -, *, /) on the stack - +
4. Put a number on the stack - 9
5. Put a operand (+, -, *, /) on the stack - *
6. Put a number on the stack - 6
7. Put a number on the stack - 3
8. Put a operand (+, -, *, /) on the stack - -
9. Put a operand (+, -, *, /) on the stack - /

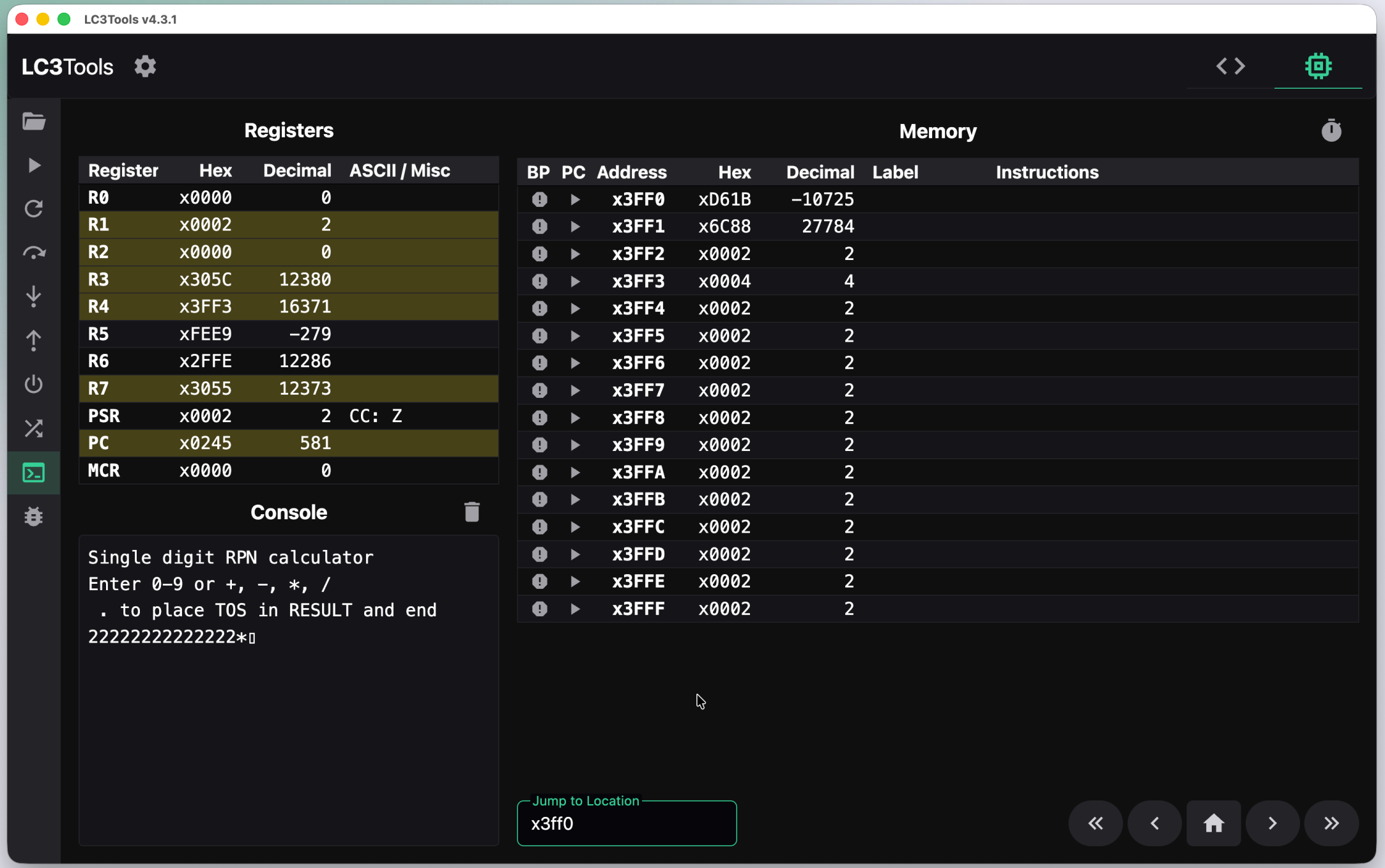
RPN	5	4	+	9	*	6	3	-	/
							3		
		4		9		6	6	3	
Stack	5	5	9	9	81	81	81	81	27

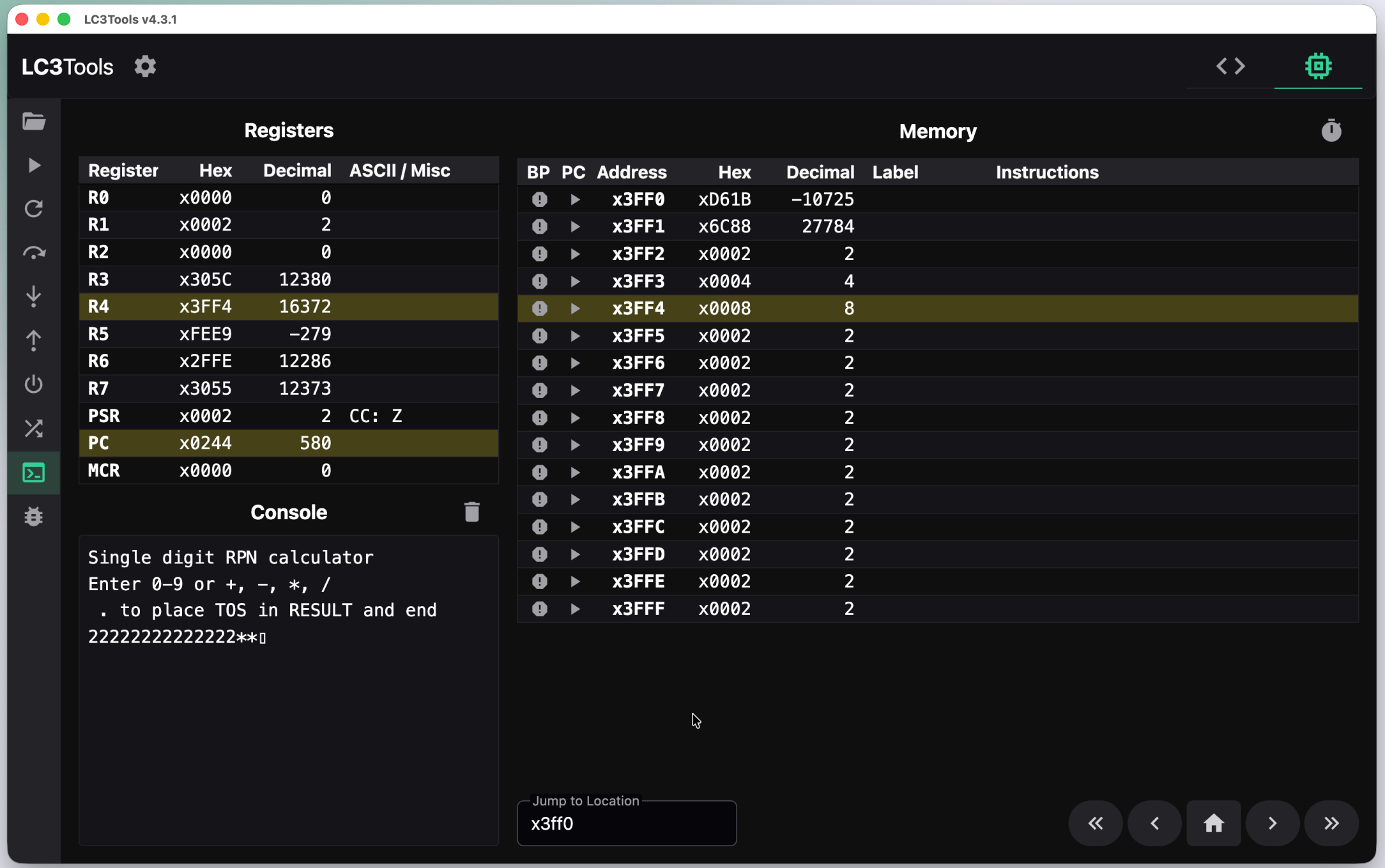
RPN Stack

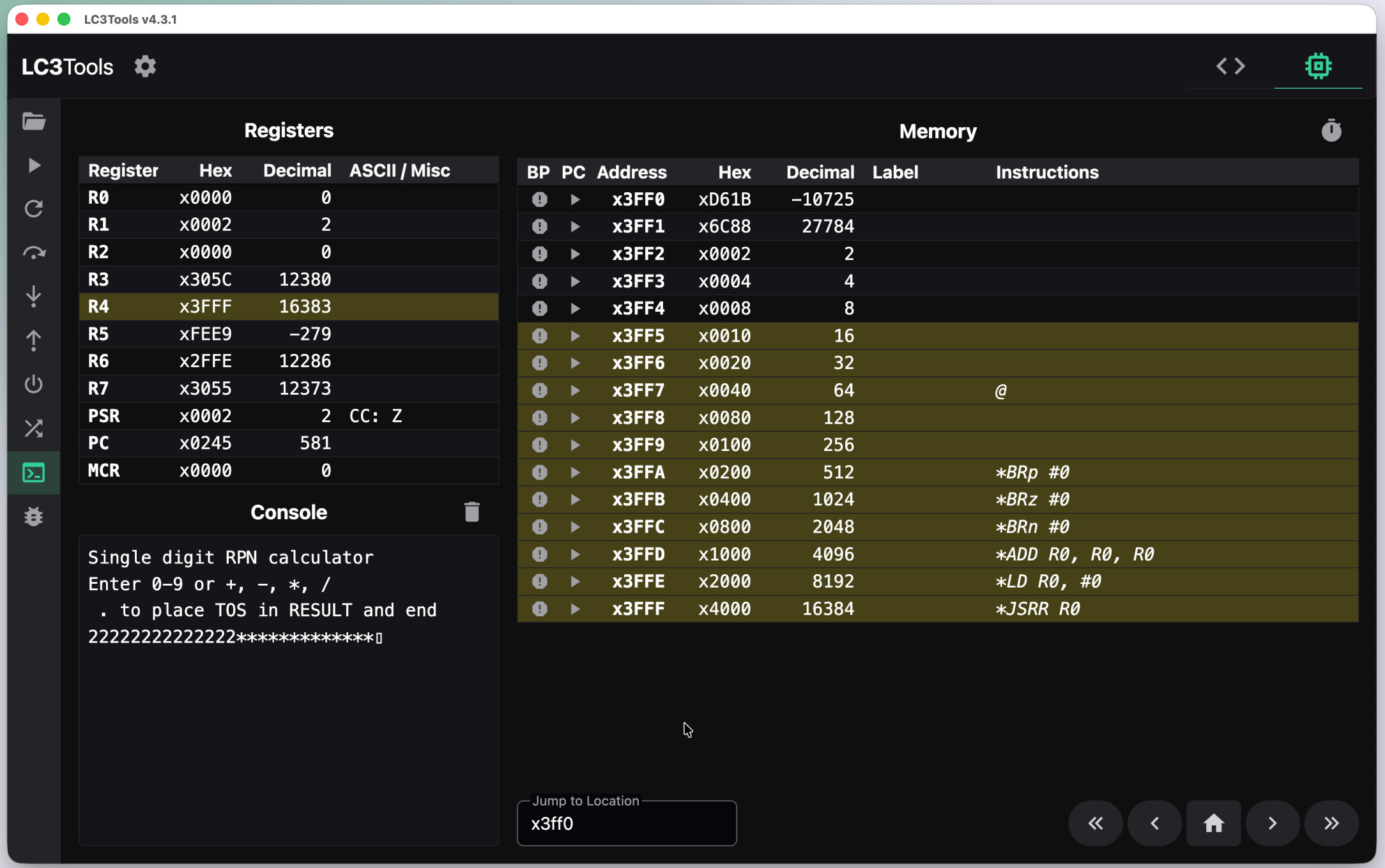
1. Operator consumes top 2 numbers on the stack
2. Result will be placed on the stack
3. There is no limit (*other than memory constraints*) to numbers on the stack
4. There must always be one less operator than numbers
5. It is helpful to have a *"print"* command such as **"."**
6. Stack doesn't delete previous entries
7. Top of the stack is at the **bottom** of memory


LC3Tools v4.3.1
LC3Tools
Registers
Register Hex Decimal ASCII / Misc
R0 x0000 0
R1 x0002 2
R2 x0000 0
R3 x305C 12380
R4 x3FFF 16383
R5 xFEE9 -279
R6 x2FFE 12286
R7 x3055 12373
PSR x0002 2 CC: Z
PC x0245 581
MCR x0000 0
Console
Single digit RPN calculator
Enter 0-9 or +, -, *, /
. to place TOS in RESULT and end
22222222222222**********
Memory
BP PC Address Hex Decimal Label Instructions
x3FF0 xD61B -10725
x3FF1 x6C88 27784
x3FF2 x0002 2
x3FF3 x0004 4
x3FF4 x0008 8
x3FF5 x0010 16
x3FF6 x0020 32
x3FF7 x0040 64 @
x3FF8 x0080 128
x3FF9 x0100 256
x3FFA x0200 512 *BRp #0
x3FFB x0400 1024 *BRz #0
x3FFC x0800 2048 *BRn #0
x3FFD x1000 4096 *ADD R0, R0, R0
x3FFE x2000 8192 *LD R0, #0
x3FFF x4000 16384 *JSRR R0
Jump to Location
x3ff0


LC3Tools v4.3.1
LC3Tools
Registers
Register Hex Decimal ASCII / Misc
R0 x0000 0
R1 x300C 12300
R2 xFFD0 -48
R3 x896D -30355
R4 x3FF2 16370
R5 xFEE9 -279
R6 x2FFE 12286
R7 x4502 17666
PSR x0002 2 CC: Z
PC x0245 581
MCR x0000 0
Console
Single digit RPN calculator
Enter 0-9 or +, -, *, /
. to place TOS in RESULT and end
22222222222222
Memory
BP PC Address Hex Decimal Label Instructions
x3FF0 xD61B -10725
x3FF1 x6C88 27784
x3FF2 x0002 2
x3FF3 x0002 2
x3FF4 x0002 2
x3FF5 x0002 2
x3FF6 x0002 2
x3FF7 x0002 2
x3FF8 x0002 2
x3FF9 x0002 2
x3FFA x0002 2
x3FFB x0002 2
x3FFC x0002 2
x3FFD x0002 2
x3FFE x0002 2
x3FFF x0002 2
Jump to Location
x3ff0


LC3Tools v4.3.1
LC3Tools
Registers
Register Hex Decimal ASCII / Misc
R0 x0000 0
R1 x0002 2
R2 x0000 0
R3 x305C 12380
R4 x3FF3 16371
R5 xFEE9 -279
R6 x2FFE 12286
R7 x3055 12373
PSR x0002 2 CC: Z
PC x0245 581
MCR x0000 0
Console
Single digit RPN calculator
Enter 0-9 or +, -, *, /
. to place TOS in RESULT and end
22222222222222*
Memory
BP PC Address Hex Decimal Label Instructions
x3FF0 xD61B -10725
x3FF1 x6C88 27784
x3FF2 x0002 2
x3FF3 x0004 4
x3FF4 x0002 2
x3FF5 x0002 2
x3FF6 x0002 2
x3FF7 x0002 2
x3FF8 x0002 2
x3FF9 x0002 2
x3FFA x0002 2
x3FFB x0002 2
x3FFC x0002 2
x3FFD x0002 2
x3FFE x0002 2
x3FFF x0002 2
Jump to Location
x3ff0


LC3Tools v4.3.1
LC3Tools
Registers
Register Hex Decimal ASCII / Misc
R0 x0000 0
R1 x0002 2
R2 x0000 0
R3 x305C 12380
R4 x3FF4 16372
R5 xFEE9 -279
R6 x2FFE 12286
R7 x3055 12373
PSR x0002 2 CC: Z
PC x0244 580
MCR x0000 0
Console
Single digit RPN calculator
Enter 0-9 or +, -, *, /
. to place TOS in RESULT and end
22222222222222**
Memory
BP PC Address Hex Decimal Label Instructions
x3FF0 xD61B -10725
x3FF1 x6C88 27784
x3FF2 x0002 2
x3FF3 x0004 4
x3FF4 x0008 8
x3FF5 x0002 2
x3FF6 x0002 2
x3FF7 x0002 2
x3FF8 x0002 2
x3FF9 x0002 2
x3FFA x0002 2
x3FFB x0002 2
x3FFC x0002 2
x3FFD x0002 2
x3FFE x0002 2
x3FFF x0002 2
Jump to Location
x3ff0


LC3Tools v4.3.1
LC3Tools
Registers
Register Hex Decimal ASCII / Misc
R0 x0000 0
R1 x0002 2
R2 x0000 0
R3 x305C 12380
R4 x3FFF 16383
R5 xFEE9 -279
R6 x2FFE 12286
R7 x3055 12373
PSR x0002 2 CC: Z
PC x0245 581
MCR x0000 0
Console
Single digit RPN calculator
Enter 0-9 or +, -, *, /
. to place TOS in RESULT and end
22222222222222**********
Memory
BP PC Address Hex Decimal Label Instructions
x3FF0 xD61B -10725
x3FF1 x6C88 27784
x3FF2 x0002 2
x3FF3 x0004 4
x3FF4 x0008 8
x3FF5 x0010 16
x3FF6 x0020 32
x3FF7 x0040 64 @
x3FF8 x0080 128
x3FF9 x0100 256
x3FFA x0200 512 *BRp #0
x3FFB x0400 1024 *BRz #0
x3FFC x0800 2048 *BRn #0
x3FFD x1000 4096 *ADD R0, R0, R0
x3FFE x2000 8192 *LD R0, #0
x3FFF x4000 16384 *JSRR R0
Jump to Location
x3ff0

RPN One More Example

Walkthrough of 123++ in RPN LC3

Begin to Develop Algorithm for SMC

Week 8 Assignment

Usage:

- Enter a number 0-9, or an operator +, -, *, or /
- Calculator will use Reverse Polish Notation to calculate
- If there is only one number on stack, operation will fail
- and program will terminate
- "." will copy the top of the RPN stack into RESULT and terminate
- If stack is empty, operation will fail and program will terminate

Programming Tips

1. Setup an initialization loop, when you have a number of constants to be converted. (*sign change*)
2. Use R4 for your *RPN* stack. Use R4 as it doesn't get over-written in system calls, making it easier to debug.
3. Keep track of register values
4. Use subroutines via jump vectors

Initialization Loop

Setup variables

```
; data section for constants
result      .fill       x0000
MULT        .fill       x2a     ; "*" multiply
PLUS        .fill       x2b     ; "+" add
...
ASCII       .fill       x30     ; ASCII to decimal conversion
count       .fill       #10     ; number of constants, MULT - ASCII
```

Initialization Loop

Change sign and store back in variable

```
            ; Setup test values for 0, 9, +, −, *, /, . and RPN stack
            LD R2, count
NEXT        LDR R0, R1, #0
            NOT R0, R0
            ADD R0, R0, #1      ; change sign of value in R0
            STR R0, R1, #0      ; save back to the original location
...
            BRnp NEXT
```

Learn the stack

Three Stack Examples:

- stack_debug_v2 - illustrates how to use debug code, similar to previous example, however, error checking moved to stack code. This simplifies the application code to just JSR PUSH or JSR POP.
- stack_prod.asm - illustrates how to use stack code which directly manipulates the stack, much more simple code. It will be very difficult to debug!
- stack_example.asm - uses comments and the stack_debug_v2 code to provide both a debug and production version. Using the stack in this method, can ensure you are able to debug, then simplify.

stack_example application code

```
START
                JSR STK_INIT    ; DEBUG: Initialize stack, REQUIRED
                ; LD R6, USER_STACK ; PROD: Initialize stack, REQUIRED

                ; Set up values to be swapped and push onto stack
                LD R0,A         ; Load a value to be pushed onto stack
                JSR PUSH        ; DEBUG: Push a value onto the stack
                ; ADD R6, R6, #-1 ; PROD: Push: Decrement
                ; STR R0, R6, #0  ; PROD: and Store

                LD R0,B         ; Load second value to be pushed onto stack
                JSR PUSH        ; DEBUG: Push a value onto the stack
                ; ADD R6, R6, #-1 ; PROD: Push: Decrement
                ; STR R0, R6, #0  ; PROD: and Store
```

PUSH Reminder

Decrement and Store (*using R4 for RPN stack*)

```
; PUSH a value, R0 into TOS
            ADD R4, R4, #-1     ; PUSH: Decrement
            STR R0, R4, #0      ;  and store
```

POP Reminder

Load and Increment (*using R4 for RPN stack*)

```
; POP two values, TOS in R2, next in R1
            LDR R2, R4, #0  ; Pop: Load
            ADD R4, R4, #1  ; and Increment
            LDR R1, R4, #0  ; Pop: Load
            ADD R4, R4, #1  ; and Increment
```

Be disciplined with registers

```
; Registers:
; R0 – primary register for operations
; R1 – temp register
; R2 – temp register
; R3 – operator vector for JSRR
; R4 – RPN stack pointer ; use R4, so it doesn't get overwritten
```

JSR vs. JSRR

What is the difference between JSR and JSRR and why is this important?

```
JSR QUEUE   ; Put the address of the instruction following JSR into R7
            ; Jump to QUEUE (PCoffset11).
JSRR R3     ; Put the address following JSRR into R7
            ; Jump to the address contained in R3.
```

Use subroutines via jump vectors

Simple algorithm

1. Input a character
2. If character is "+", subroutine PLUS
3. If character is "-", subroutine MINUS
4. If character is "x", subroutine MULT
5. If character is "/", subroutine DIVD
6. else, subroutine ERROR

Coding with JSR

```
; if operator identified, get subroutine vector
chk_ad      LD R2, plus
            ADD R2, R0, R2
            BRnp chk_mi
            LEA R3, op_ad       ; get ADD subroutine vector
...
; load registers and goto subroutine via vector
            LDR R1, R4, #0  ; Pop: Load
            ADD R4, R4, #1  ; and Increment
            JSRR R3             ; call ADD subroutine with
                                ; arguments in registers
            BR loop             ; on return, get a new char
```

Excellent Detailed Description of RPN

- Reverse Polish Notation